Pinelabs Fin

Pine Labs payment integration for Frappe & ERPNext — take terminal payments or send hosted payment links straight from any invoice, with the Payment Entry created automatically when the customer pays.



Main features

Pay on Machine (terminal): - Card and UPI on a physical Pine Labs Cloud terminal. - Foreground "Waiting for terminal" modal + background cron reconciliation. - Auto-creates the ERPNext Payment Entry on success.

Send Payment Link (online): - Plural hosted checkout — card, UPI, netbanking, wallets. - Pine Labs delivers the link by SMS and email automatically. - Webhook + 1-minute reconcile cron mark the invoice **Paid** when the customer pays.

Marketplace-clean integration: - Zero custom fields on Sales Invoice, POS Invoice, Payment Entry, Customer or Contact. - All payment state lives in a dedicated `Pinelabs Transaction` doctype, linked via Dynamic Link. - Supports Sales Invoice, POS Invoice, Payment Request, and any custom payable doctype.

Operations: - Multi-terminal routing — pick a terminal automatically by user, warehouse, or POS profile. - Every API exchange is captured on the Pinelabs Transaction record (request, response, error). - Whitelisted Python + JS APIs to drive the same flows from your own buttons / hooks / scripts.

How to Install

```
cd ~/frappe-bench
bench get-app https://github.com/<owner>/pinelabs_fin
bench --site <your-site> install-app pinelabs_fin
bench --site <your-site> migrate
bench build
bench restart
```

Upgrade later:

```
cd ~/frappe-bench/apps/pinelabs_fin
git pull
cd ~/frappe-bench
bench --site <your-site> migrate
```

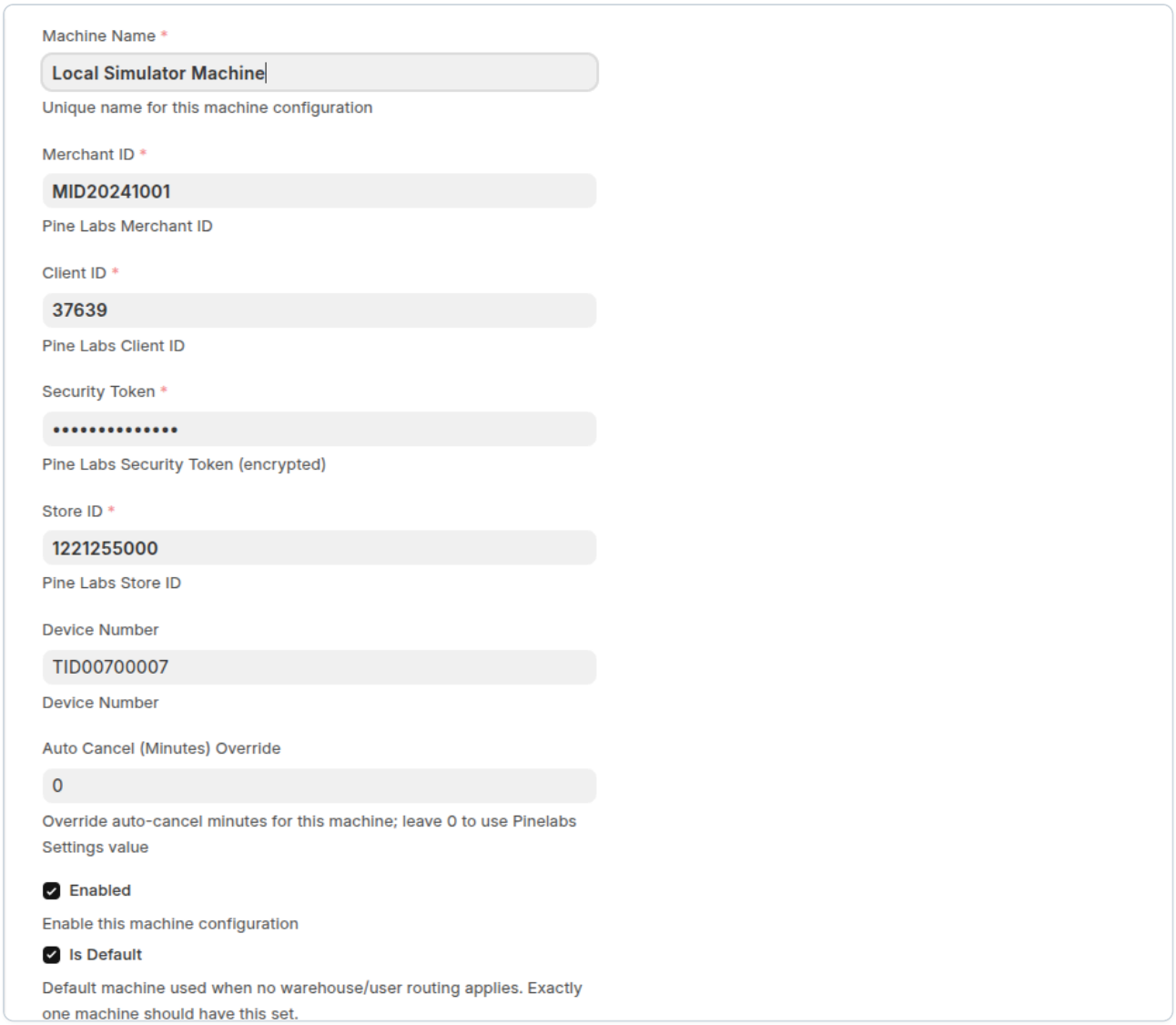
Migrations are idempotent — `bench migrate` is safe to run any number of times.

Setup and Use

Everything is configured in a single doctype: **Pinelabs Settings**. Open the desk Awesome Bar and type "Pinelabs Settings".

Configure your Pine Labs machine

Open **Pinelabs Settings** → **Machines** and add one row per terminal.



Field	What to put
Machine Name	Any short label (e.g. "Front counter")
Client ID, **Security Token**, **Merchant ID**, **Store ID**, **Terminal ID**	From your Pine Labs onboarding email
Enabled	✅
Is Default	✅ on **exactly one** machine

Save. The app can now talk to your terminal.

Configure Plural (for Send Payment Link)

Skip this if you only use **Pay on Machine**. In **Pinelabs Settings → Plural API Configuration**:

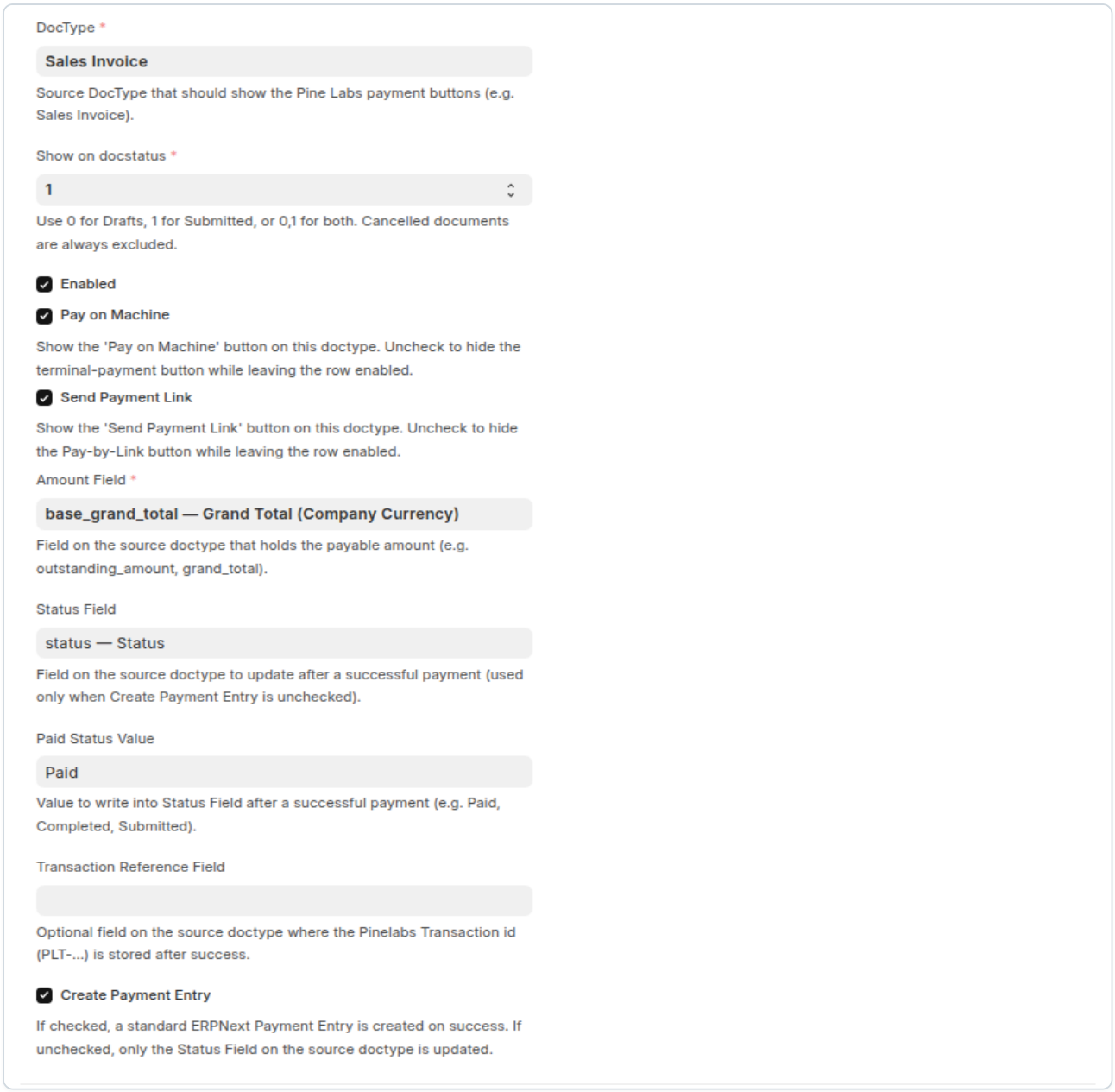
Field	What to put
Enable Payment Links	✅
Plural Client ID / **Plural Client Secret**	From your Pine Labs Plural dashboard
Webhook Secret (HMAC)	A random string ≥ 16 characters — you'll set the **same** value on the Plural dashboard
Plural API Base URL	Leave blank for sandbox. For production use `https://api.pluralpay.in`

Click **Test Plural Connection** to verify. Then on the Plural dashboard, register the webhook URL:

```
https://<your-erpnext-host>/api/method/pinelabs_fin.api.webhook.handle_webhook
```

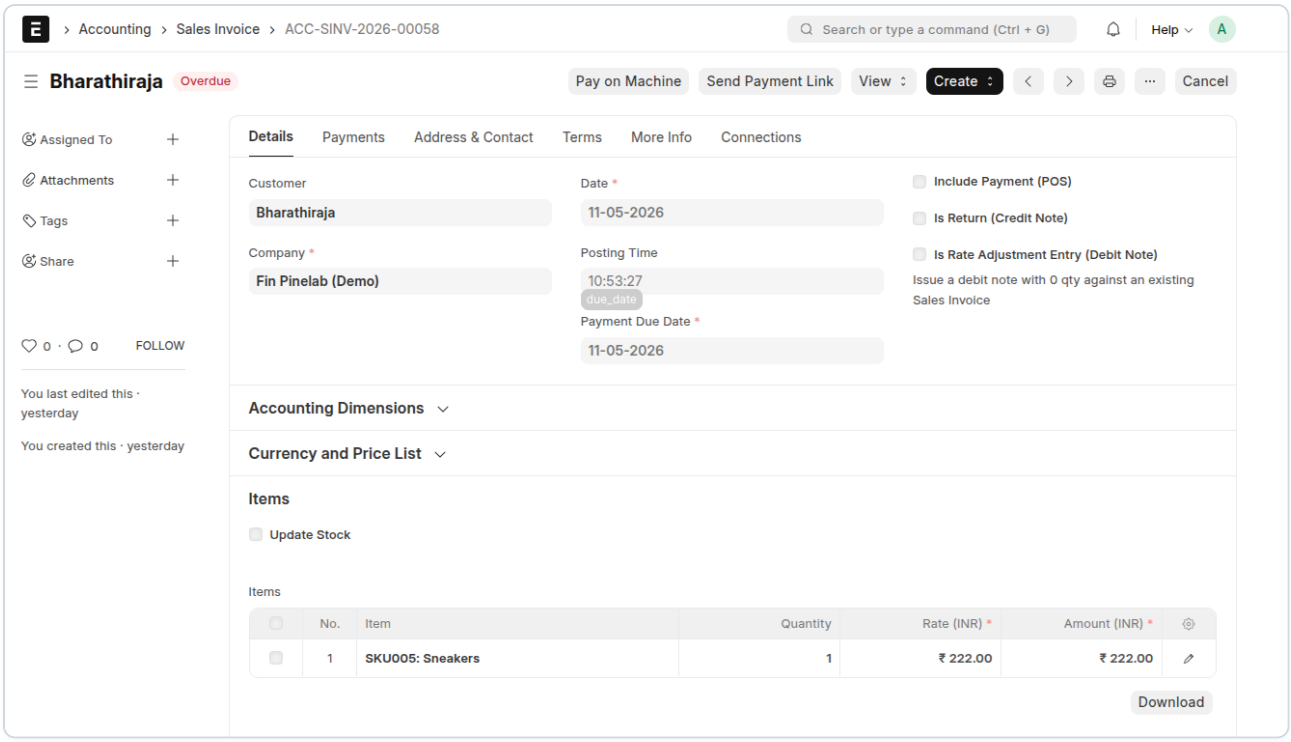
Configure Payable Documents

In **Pinelabs Settings** → **Payable Documents**, add one row per ERPNext doctype that should show the payment buttons.



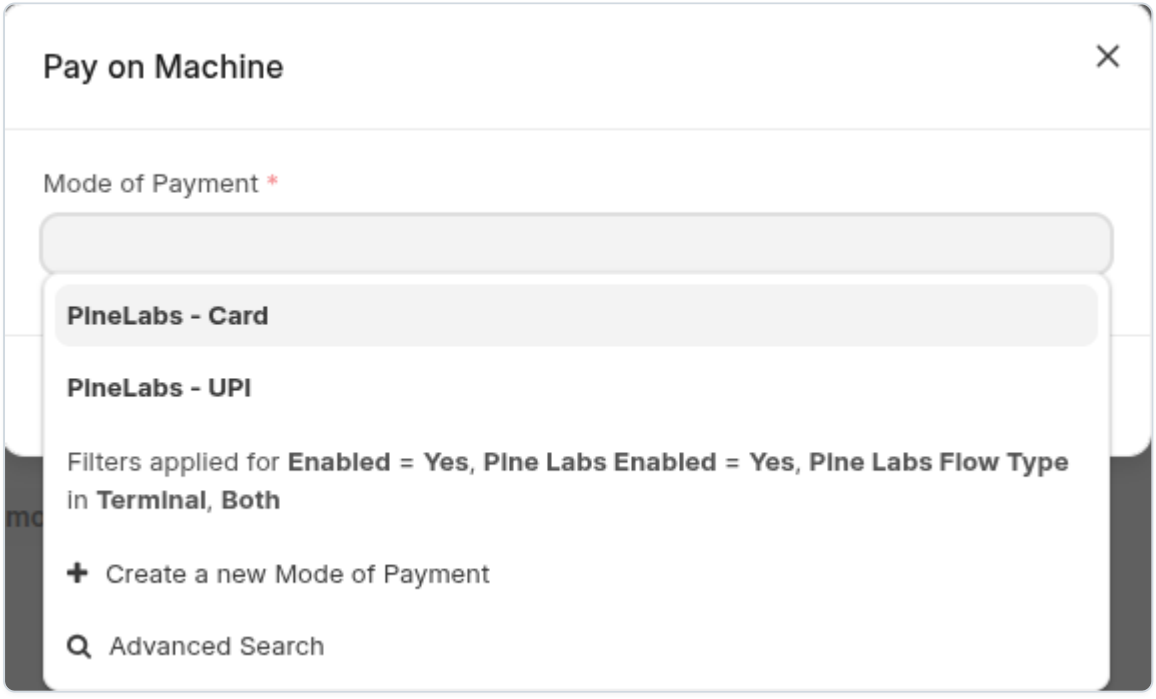
Field	What to put
DocType	Source doctype (e.g. `Sales Invoice`)
Enabled	✅
Pay on Machine	✅ to show the terminal button
Send Payment Link	✅ to show the payment-link button
Amount Field	Field holding the payable amount (e.g. `base_grand_total`)
Create Payment Entry	✅ for Sales Invoice / POS Invoice / Payment Request

Save. Open any submitted record of that doctype — the two buttons appear at the top.

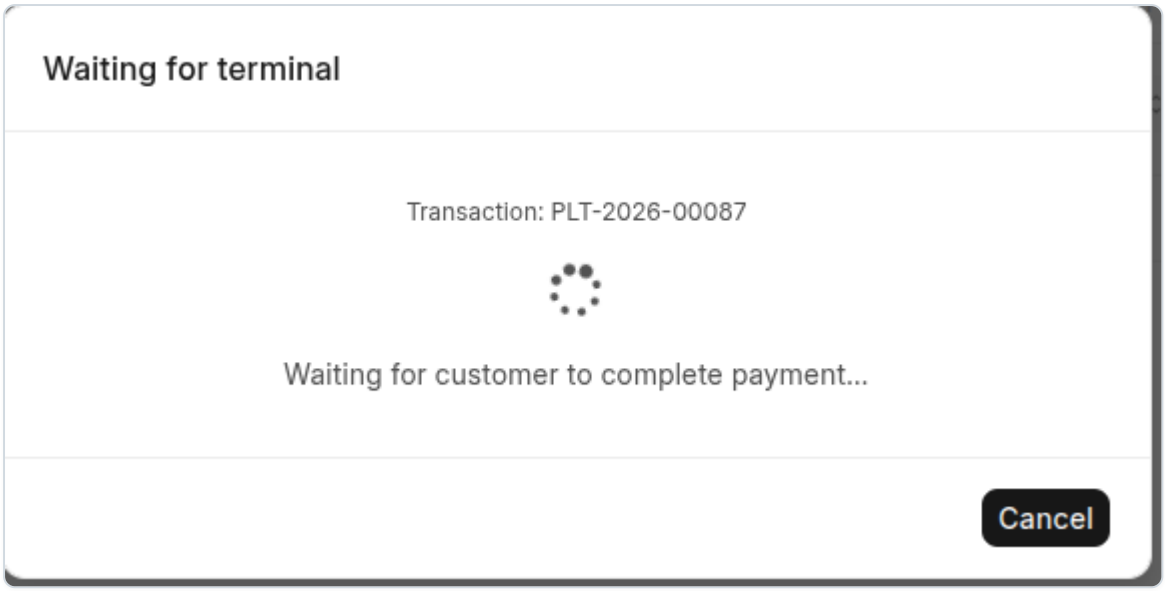


Pay on Machine — when the customer is at the counter

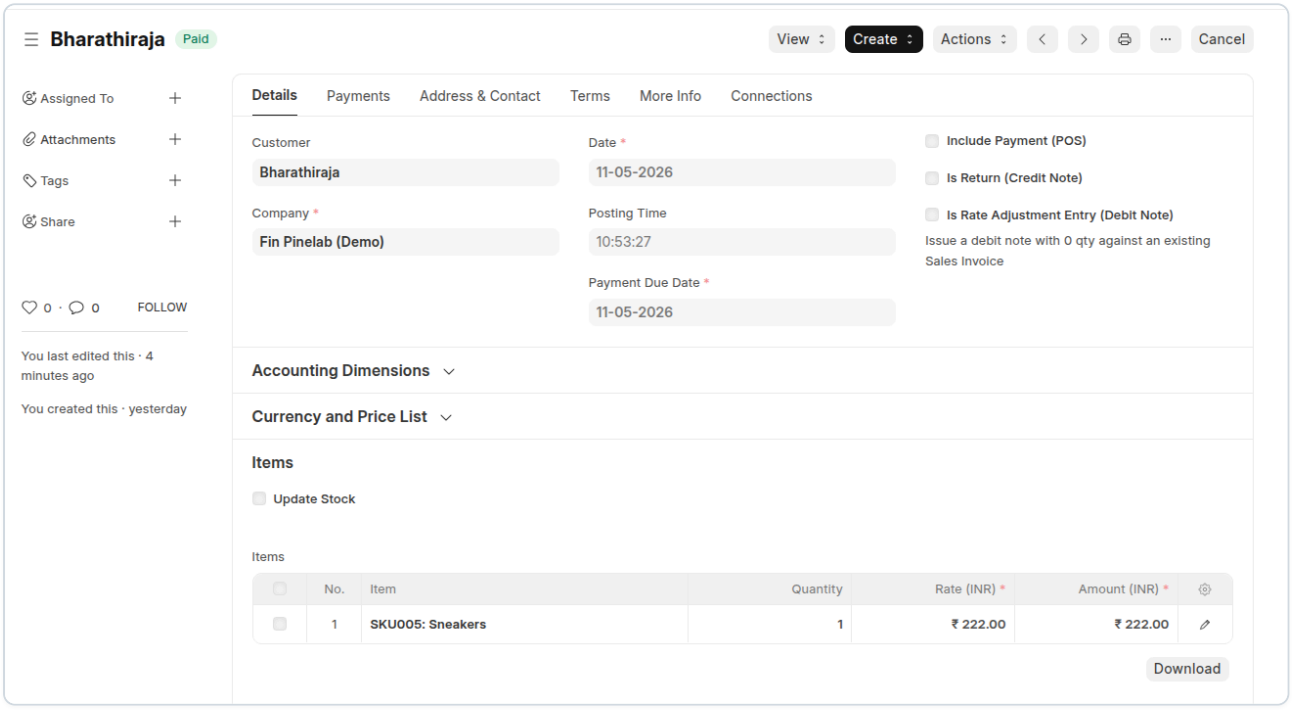
1. Open the submitted invoice.
2. Click **Pay on Machine**.
3. Pick **PineLabs - Card** or **PineLabs - UPI** and click **Start Payment**.



4. The "Waiting for terminal" window appears and the Pine Labs machine beeps with the bill.



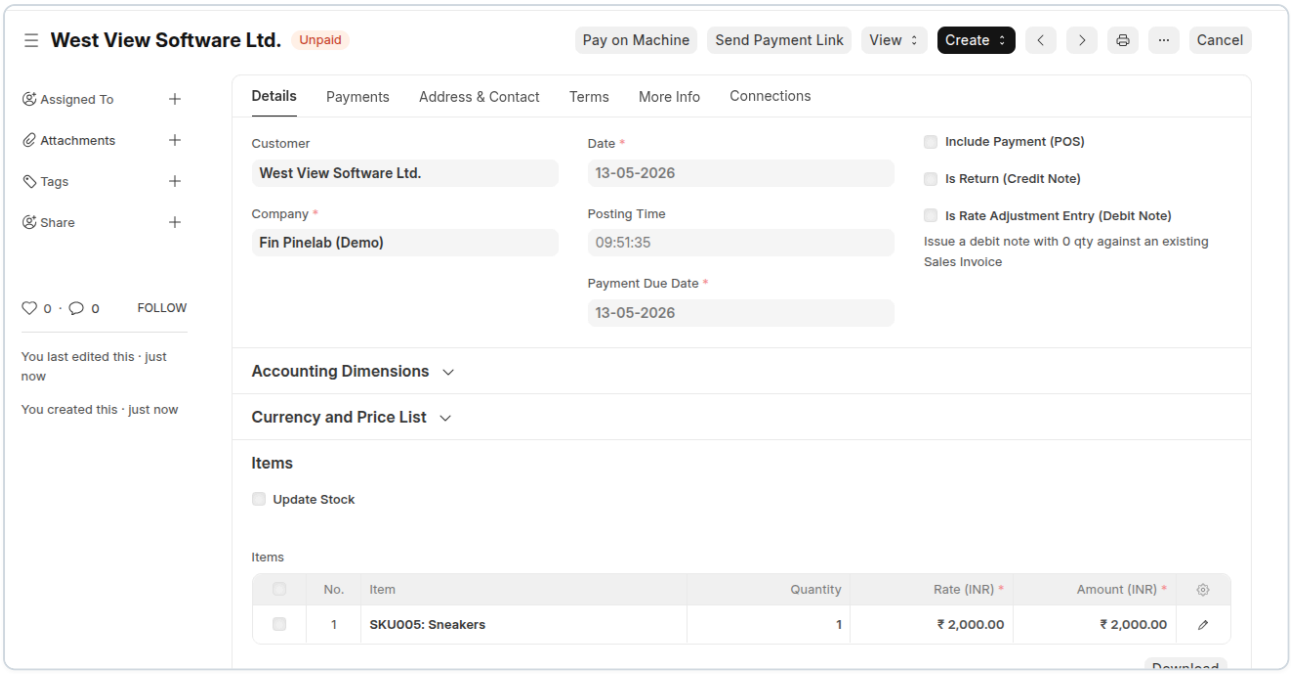
5. Customer taps card or scans UPI. The invoice flips to **Paid** and a Payment Entry is created automatically.



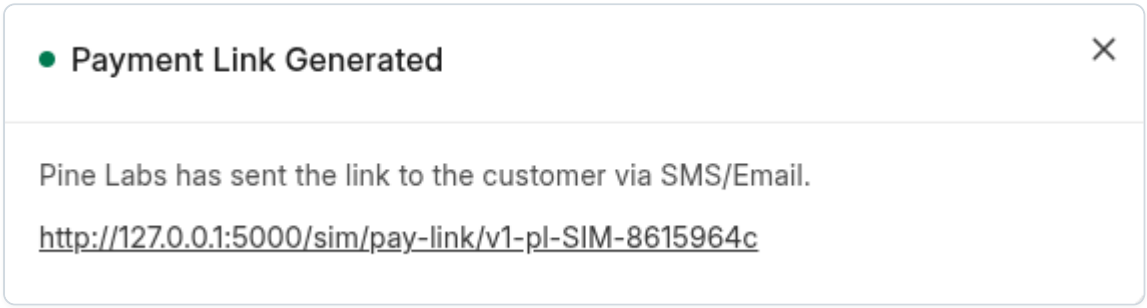
End-to-end takes ~5–10 seconds.

Send Payment Link — when the customer is elsewhere

1. Open the submitted invoice. Make sure it has an email or mobile number resolvable to the customer.
2. Click **Send Payment Link**.



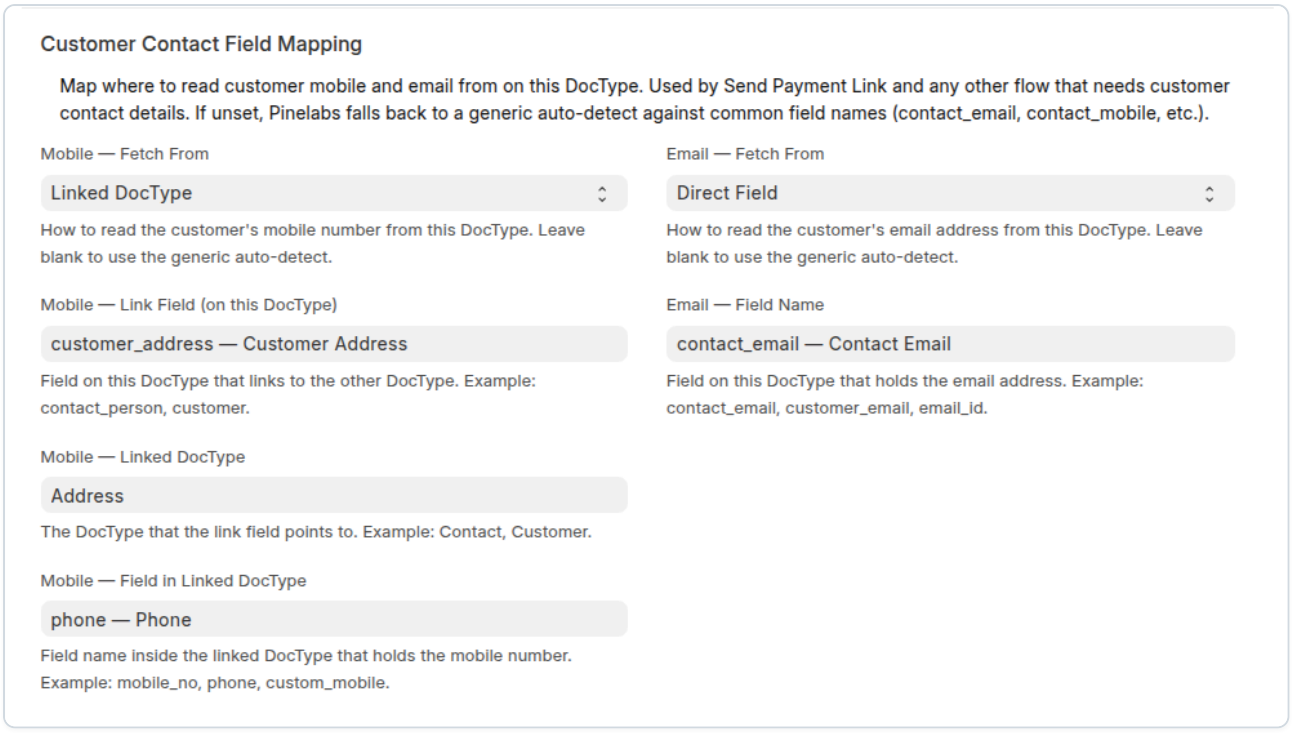
3. A green popup shows the generated link.



4. Pine Labs delivers it to the customer by SMS and email.
5. When the customer pays, the invoice flips to **Paid** within a minute.

Customer Contact Field Mapping

The app needs an email / mobile to give Plural. In each Payable Documents row, **Customer Contact Field Mapping** tells the app where to find them.

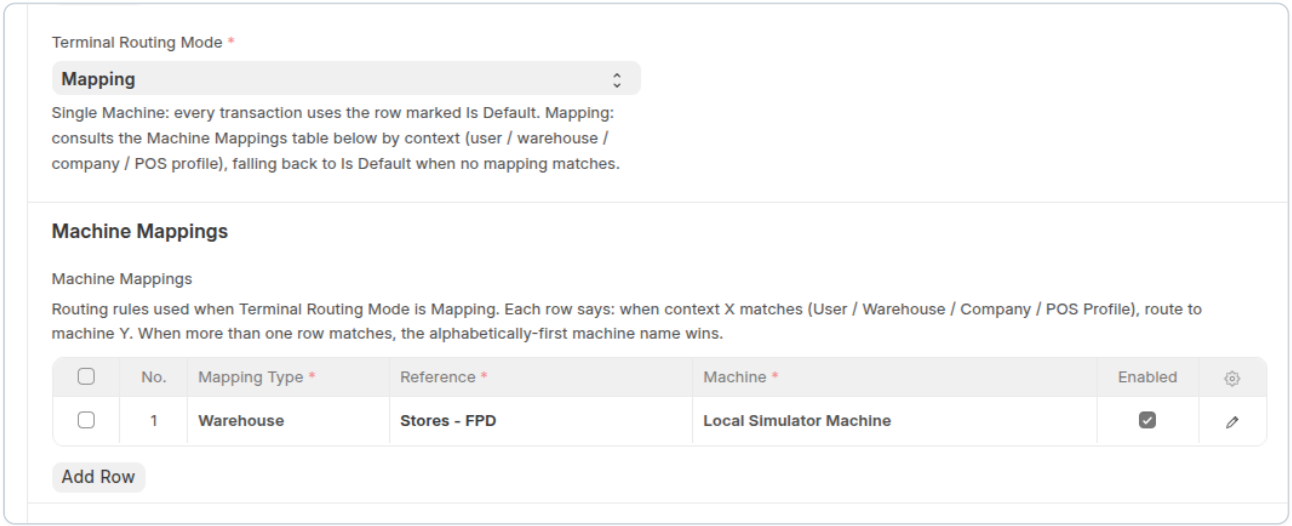


- **Direct Field** — the value sits on the source doctype itself (e.g. `contact_email` on Sales Invoice).
- **Linked DocType** — the value sits on a linked doctype (e.g. mobile on Address via `customer_address` → field `phone`).

Mobile and Email are independent — pick whichever mode fits each. Leave both blank to let the app auto-detect common field names (`mobile_no`, `email_id`, `contact_email`, …).

Optional — multi-terminal routing

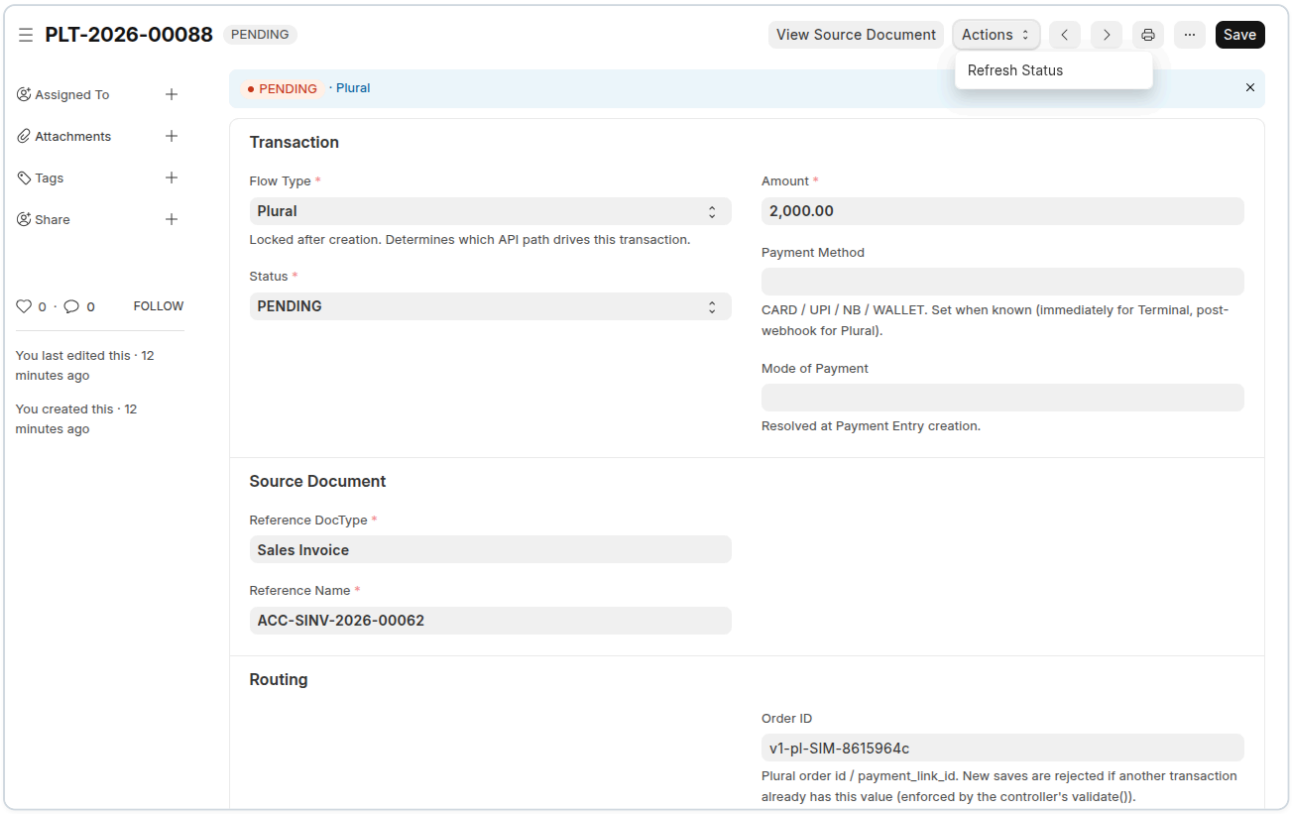
Two or more terminals? Switch **Terminal Routing Mode** to `Mapping` and add rules.



The app picks the right terminal automatically based on the user at the till, the warehouse on the invoice, or the active POS profile. No rule matches → falls back to the default machine.

Track every payment

Every payment attempt lands on one **Pinelabs Transaction** record. Status moves through `INITIATED → PENDING → SUCCESS / FAILED / CANCELLED`. The record carries the full request and response payloads for debugging.



Click **Actions → Refresh Status** to ask Pine Labs on demand. Otherwise a 1-minute scheduler reconciles in the background.

API

Three whitelisted Python entry points — callable from JS or REST.

```
# Pay on Machine
pinelabs_fin.api.terminal.initiate_terminal_payment(
    reference_doctype,    # required — e.g. "Sales Invoice"
    reference_name,       # required — e.g. "ACC-SINV-2026-00054"
    mode_of_payment,      # required — a PineLabs-enabled Mode of Payment
    amount=None,          # optional — defaults to source doc's outstanding
)

# Send Payment Link
pinelabs_fin.api.plural.initiate_payment_link(
    reference_doctype,        # required
    reference_name,           # required
    flow="all_methods",       # optional — "all_methods" / "upi_only" / "card_on
    expiry_minutes=None,      # optional — defaults to 1440 (24 h)
    customer_email=None,      # optional — inline override
    customer_mobile=None,     # optional — inline override
)

# Poll a Pay-on-Machine transaction
pinelabs_fin.api.terminal.check_terminal_status(
    transaction_name,    # required — the PLT-... from initiate_terminal_payment
)
```

Plus two JS helpers on `frappe.pinelabs.*` that wrap the API and open the same UI the built-in buttons use:

```
frappe.pinelabs.start_terminal_payment(
  { reference_doctype: frm.doctype, reference_name: frm.docname, mode_of_payment
  { on_settled: () => frm.reload_doc() },
);

frappe.pinelabs.start_payment_link(
  { reference_doctype: frm.doctype, reference_name: frm.docname, flow: "all_meth
  { on_link: () => frm.reload_doc() },
);
```

Limitations

- **Refunds** are not initiated by the app — use Pine Labs' own refund flow, then reflect the change in ERPNext manually.
- **Currency** — only INR.

- **Automatic Payment Entry** works for Sales Invoice, POS Invoice and Payment Request. Custom doctypes use the status-field mapping path instead.

Dependencies

- Frappe v15
- ERPNext v15
- Python 3.10+
- MariaDB 10.6+ with InnoDB

License

MIT — see license.txt.

Built with Frappe · by Finstein